

Day 4: Advanced OOP Concepts (12-June-2026, Friday)

Session Time: 9:00 AM – 4:30 PM (Lunch Break: 1:00 PM – 2:00 PM, Tea Breaks: 11:00 AM & 3:00 PM)

Session Objectives

By the end of Day 4, learners will be able to:

- Implement inheritance using `extends` and understand constructor chaining with `super()`.
 - Override methods and apply runtime polymorphism (dynamic method dispatch).
 - Differentiate between method overloading (compile-time) and overriding (runtime).
 - Use abstract classes and abstract methods to define partial abstractions.
 - Implement interfaces to achieve multiple inheritance and contract-based programming.
 - Solve LeetCode problems: Valid Parentheses (using stack) and Single Number (using XOR).
 - Complete HackerRank challenges on Inheritance and Abstract Classes.
 - Design a **Banking Transaction Processing System** with polymorphic account types.
-

1. Inheritance

1.1 Definition

- Inheritance allows a class (child/subclass) to acquire properties and behaviors of another class (parent/superclass).
- Key words: `extends` (for classes), `implements` (for interfaces).
- Java supports **single inheritance** for classes (one direct parent) but **multiple inheritance through interfaces**.

1.2 Syntax

```
class ChildClass extends ParentClass {  
    // additional fields and methods  
}
```

1.3 What is Inherited?

- `public` and `protected` members (fields, methods, nested classes).
- `default` (package-private) members if subclass is in same package.
- `private` members are **not inherited** but can be accessed indirectly via `public/protected` getters or methods.
- Constructors are **not inherited** but can be invoked via `super()`.

1.4 `super` Keyword

- Refers to the immediate parent class object.
- Uses:
 - Access parent's field: `super.field` (when shadowed).
 - Call parent's method: `super.method()`.
 - Call parent's constructor: `super(parameters)` – must be first statement in child constructor.

1.5 Constructor Chaining

- Every subclass constructor must call a parent constructor (explicitly or implicitly).
- If no `super()` call is written, compiler inserts `super()` (no-arg constructor of parent).
- If parent has no no-arg constructor, child must explicitly call `super(parameters)`.
- The chain continues up to `Object` class.

1.6 Inheritance Hierarchy

- All classes implicitly extend `java.lang.Object` (root of hierarchy).
- Methods from `Object`: `toString()`, `equals()`, `hashCode()`, `getClass()`, etc.

1.7 When to Use Inheritance?

- **IS-A relationship:** A `SavingsAccount` IS-A `BankAccount`.
- Code reuse – child can reuse parent's methods.
- Polymorphism – allowing different behaviors via method overriding.

2. Method Overriding

2.1 Definition

- Subclass provides a specific implementation of a method already defined in parent.
- Must have same name, return type (or covariant return type), and parameter list.
- Access modifier cannot be more restrictive (can be broader).

- Cannot override `final` or `static` methods.

2.2 `@Override` Annotation

- Optional but recommended – instructs compiler to check that method correctly overrides.
- Prevents mistakes (e.g., misspelled method name or wrong parameter types).

2.3 Rules for Overriding

Rule	Description
Method signature	Must match exactly (name + parameter types + order)
Return type	Must be same or covariant (subclass of original return type)
Access modifier	Cannot be more restrictive; can be less restrictive (e.g., protected → public)
<code>final</code> methods	Cannot be overridden
<code>static</code> methods	Cannot be overridden (can be hidden – different concept)
<code>private</code> methods	Not inherited, so cannot be overridden
Exceptions	Can throw fewer or narrower (subclass) exceptions, not broader

2.4 Overriding vs Overloading

Feature	Overloading	Overriding
Relationship	Same class or subclass (but not required)	Between parent and child
Method name	Same	Same
Parameters	Must differ	Must be identical
Return type	Can differ	Must be same or covariant
Binding	Compile-time (static)	Runtime (dynamic)
Annotations	None specific	<code>@Override</code>

3. Runtime Polymorphism (Dynamic Method Dispatch)

3.1 Definition

- Ability of a single reference variable to call different method implementations based on

actual object type at runtime.

- Achieved via **method overriding** and **upcasting**.

3.2 Upcasting

- Assigning a subclass object to a superclass reference.
- Example: `Parent p = new Child();`
- Allowed because a child IS-A parent.

3.3 Dynamic Method Dispatch

- When an overridden method is called through a superclass reference, the JVM determines which version to execute based on the **actual object type** (not reference type).
- Decided at runtime, hence "runtime polymorphism".

3.4 Downcasting

- Explicitly converting a superclass reference back to subclass type.
- Syntax: `Child c = (Child) p;` (only if p actually points to a Child object).
- `instanceof` operator checks type before downcasting to avoid `ClassCastException`.

3.5 Example Scenario (Conceptual)

- `Animal` class with `makeSound()`.
 - `Dog` and `Cat` override `makeSound()`.
 - `Animal ref = new Dog(); ref.makeSound();` → prints "Woof" (Dog's version).
 - At compile time, compiler only knows `Animal` has `makeSound()`; at runtime, JVM sees the object is `Dog`.
-

4. Abstraction – Abstract Classes

4.1 Definition

- Abstraction hides implementation details and shows only essential features.
- Achieved via **abstract classes** and **interfaces**.

4.2 Abstract Class

- Declared with `abstract` keyword.
- Cannot be instantiated (cannot use `new`).
- May contain:
 - Abstract methods (no body, just declaration)

• Abstract methods (no body, just declaration).

- Concrete methods (with implementation).
- Fields, constructors, static members.
- Subclass must implement all abstract methods (or become abstract itself).

4.3 Abstract Method

- Declared without a body: `public abstract void doSomething();`
- Forces subclasses to provide implementation.

4.4 When to Use Abstract Class?

- You want to share code among several closely related classes.
- You expect subclasses to have common fields or methods.
- You want to declare non-static or non-final fields (interfaces only have static final fields implicitly).

4.5 Concrete Class

- A class that implements all abstract methods and can be instantiated.

5. Interfaces

5.1 Definition

- A reference type in Java that is a **contract** of what a class can do, without specifying how.
- Declared with `interface` keyword.
- All methods are implicitly `public abstract` (before Java 8).
- Fields are implicitly `public static final`.

5.2 Implementing an Interface

- A class uses `implements` keyword.
- Must implement all abstract methods (or be abstract).
- Can implement multiple interfaces: `class C implements I1, I2, I3`

5.3 Interface vs Abstract Class

Feature	Abstract Class	Interface
Multiple inheritance	No (single)	Yes (multiple)
Instance variables	Yes	No (only static final)

Constructors	Yes	No
Access modifiers	All (public, protected, private)	Only public (default if none)
Method implementation	Can have concrete methods	All abstract (before Java 8)
When to use	IS-A relationship with shared code	CAN-DO or capability contract

5.4 Java 8+ Interface Features

- **Default methods:** `default` keyword – provide implementation in interface; classes can override.
 - Solves interface evolution problem.
- **Static methods:** `static` methods in interfaces (called using interface name).
- **Private methods** (Java 9+) – helper methods for default/static methods.

5.5 Functional Interfaces

- Interface with exactly **one** abstract method.
- Can be used with lambda expressions (Day 9).
- Annotation `@FunctionalInterface` (optional, but recommended).

6. LeetCode Problems – Approach Discussion

6.1 Valid Parentheses

- **Problem:** Given a string containing `'('`, `)'`, `'{'`, `'}'`, `'['`, `']'`, determine if brackets are valid (correctly closed and nested).
- **Key concept:** Use a **stack** data structure (LIFO).
- **Algorithm:**
 - Iterate each character.
 - If opening bracket, push to stack.
 - If closing bracket:
 - If stack empty → invalid.
 - Pop top – must be matching opening bracket.
 - At end, stack must be empty.
- **Time complexity:** $O(n)$, space $O(n)$.
- **OOP relevance:** Stack can be implemented using `java.util.Stack` (extends `Vector`) or `ArrayDeque`.

6.2 Single Number

- **Problem:** Given a non-empty array of integers where every element appears twice except one, find the single one.
 - **Constraints:** Linear time, constant extra space.
 - **Key insight:** XOR (`^`) operation properties:
 - `a ^ a = 0`
 - `a ^ 0 = a`
 - XOR is commutative and associative.
 - **Algorithm:** Initialize result = 0; XOR all elements. Duplicates cancel out, remaining is the single number.
 - **Time complexity:** O(n), space O(1).
 - **No OOP directly** but demonstrates bitwise operator usage.
-

7. HackerRank Problems – Approach Discussion

7.1 Java Inheritance (I and II)

- **Task (Inheritance I):** Create a class `Animal` with a method `walk()` . Create subclass `Bird` that extends `Animal` , adds `fly()` and `sing()` .
- **Task (Inheritance II):** Create class `Arithmetic` with method `add(int a, int b)` . Subclass `Adder` inherits `add` without overriding.
- **Skills practiced:** `extends` , method inheritance, calling inherited methods.

7.2 Java Abstract Class

- **Task:** Create abstract class `Book` with an abstract method `void setTitle(String s)` . Concrete class `MyBook` implements `setTitle` and stores title.
 - **Skills practiced:** Abstract class declaration, abstract method implementation, creating concrete subclass.
-

8. Corporate Use Case: Banking Transaction Processing System

8.1 Problem Context

A bank requires a transaction processing system that handles different types of bank accounts:

- **SavingsAccount** – interest rate, minimum balance requirement.

- **CurrentAccount** – overdraft facility.
- **FixedDepositAccount** – lock-in period, penalty for early withdrawal.

All accounts support common operations: `deposit()`, `withdraw()`, `getBalance()`, `calculateInterest()`. However, each account type implements these differently. Additionally, the system should allow processing a list of transactions polymorphically without knowing the exact account type at compile time.

8.2 Application of Day 4 Topics

Requirement	Java Feature Used
Common account behavior	Abstract class <code>BankAccount</code> with abstract methods <code>withdraw()</code> , <code>calculateInterest()</code>
Shared fields (accountNumber, balance)	Abstract class with concrete fields and constructor
Savings specific rules	Subclass <code>SavingsAccount</code> overrides <code>withdraw()</code> (check min balance)
Current overdraft	Subclass <code>CurrentAccount</code> overrides <code>withdraw()</code> (allow negative up to limit)
Fixed deposit penalty	Subclass <code>FixedDeposit</code> overrides <code>withdraw()</code> (penalty logic)
Process transactions without type knowledge	Runtime polymorphism – <code>BankAccount</code> reference pointing to any subclass
Additional capability like <code>LoanEligible</code>	Interface <code>LoanEligible</code> with method <code>canTakeLoan()</code>
Multiple capabilities	A class can implement multiple interfaces (e.g., <code>SavingsAccount</code> implements <code>LoanEligible</code> , <code>Taxable</code>)
Ensure all accounts have interest calculation	Abstract method <code>calculateInterest()</code> forces implementation
Transaction logging (default behavior)	Default method in interface <code>TransactionLogger.logTransaction()</code> (Java 8)
Static utility method to transfer money	Static method in interface or abstract class

8.3 Class Hierarchy Diagram (Conceptual)

```

BankAccount (abstract)
- accountNumber: String
- balance: double
  
```



```

+ deposit(amount)
+ withdraw(amount) → abstract
+ getBalance()
+ calculateInterest() → abstract
    |
    +-----+-----+
    |           |           |
SavingsAccount CurrentAccount FixedDeposit
- minBalance    - overdraftLimit - lockPeriodMonths
+ withdraw()    + withdraw()      + withdraw()
+ calculateInterest() + calculateInterest() + calculateInterest()
    |           |
    |           | implements LoanEligible
    +-----+
    |
implements LoanEligible

```

8.4 Polymorphism in Action

- A transaction processor can loop through a list of `BankAccount` objects:
 - Call `withdraw(amount)` – the correct subclass version executes.
 - Call `calculateInterest()` – each account computes its own interest.
- Adding a new account type (e.g., `SalaryAccount`) requires no changes to the transaction processing logic – just extend the abstract class.

8.5 Interface Example – LoanEligible

```

public interface LoanEligible {
    boolean canTakeLoan();
    double getMaxLoanAmount(); // abstract
}

```

- `SavingsAccount` implements this: eligibility based on balance and account age.
- `CurrentAccount` may also implement with different rules.
- A bank manager module can accept any `LoanEligible` object without caring whether it's a savings or current account.

8.6 Discussion Questions

- Why make `withdraw()` abstract instead of providing a default implementation? (Each account has different rules – cannot standardize.)
- How would you implement `calculateInterest()` for Savings vs Fixed Deposit? (Savings: daily/monthly compounding; Fixed: fixed rate unless broken.)
- What if we need a new method `applyTax()` that differs by account type? Add abstract method in `BankAccount`? (Yes – but careful: not all accounts may be taxable. Better to create interface `Taxable`.)

```
interface Taxable {}
```

- How does runtime polymorphism help when processing a million transactions? (Code is maintainable; new account types don't break existing loops.)

9. Hands-on Exercises (Without Code Lines)

9.1 Inheritance – Constructor Chaining Tracing

Given class hierarchy:

- A (no-arg constructor prints "A")
 - B extends A (constructor with int x prints "B" + x)
 - C extends B (constructor with int y calls super(y*2) prints "C" + y)
- What is printed when we create `new C(5)` ? Trace step by step.

9.2 Overriding vs Hiding

- Parent has `static void show()` and `void display()`.
- Child defines `static void show()` (hiding) and overrides `display()`.
- Given `Parent p = new Child(); p.show(); p.display();` Which methods are called? (Static methods are not polymorphic – parent's static called; display is overridden – child's display called.)

9.3 Abstract Class Design

Describe (in English) the abstract class `Shape` with:

- Fields: `color` (String), `filled` (boolean).
- Abstract method: `getArea()`.
- Concrete method: `getColor()`.

Then describe a `Circle` subclass that implements `getArea()` using radius.

9.4 Multiple Interfaces Scenario

Explain how you would design classes for:

- `Robot` that implements `Movable` (method `move()`) and `Speakable` (method `speak()`).
- What if both interfaces have a `default` method with same signature? (Compile error – must override in class.)

9.5 Valid Parentheses Dry Run

Given the string `"{[(())]}"`, simulate the stack operations step by step. Then simulate `"(()){}"` and identify when it fails.

10. Quiz & Quick Check (Conceptual)

1. Which keyword is used to call the parent class constructor?

- (a) `this`
- (b) `parent`
- (c) `super`
- (d) `base`

2. What is the output when `Animal a = new Dog(); a.makeSound();` if `Dog` overrides `makeSound()` ?

- (a) Animal's `makeSound`
- (b) Dog's `makeSound`
- (c) Compilation error
- (d) Runtime error

3. Which of the following is true about an abstract class?

- (a) It can be instantiated.
- (b) It cannot have constructor.
- (c) It can have both abstract and concrete methods.
- (d) All methods must be abstract.

4. How many interfaces can a class implement?

- (a) One
- (b) Two
- (c) Unlimited
- (d) Only if there is no abstract class

5. Given `int x = 5 ^ 3;` what is the value of x?

- (a) 8
- (b) 6
- (c) 2
- (d) 1

Answers: 1-c, 2-b, 3-c, 4-c, 5-b (5=101, 3=011, XOR=110 = 6)

11. Group Activity – Pair Programming Simulation

- **Duration:** 20 minutes (during hands-on slot).
 - **Task:** Design a **transactions processor** for the banking system:
 - Define an abstract class `BankAccount` with `withdraw(double amount)`, `deposit(double amount)` (concrete), and `calculateInterest()` (abstract).
 - Create two subclasses: `SavingsAccount` (withdraw fails if balance falls below 500) and `CurrentAccount` (overdraft limit 5000).
 - Write polymorphic code (pseudocode) that processes an array of accounts, calling `withdraw` and `calculateInterest` without using `instanceof`.
 - **Outcome:** Each pair explains how runtime polymorphism allows the processor to work for any future account type.
-

12. Additional Resources & Cheat Sheets

- **Inheritance rules:** Single class inheritance, multiple interface inheritance.
 - **super usage:** `super()` must be first statement in constructor; `super.method()` can be anywhere.
 - **Overriding checklist:** Same name, same parameters, same or covariant return, not more restrictive access, not final/static.
 - **Abstract vs Interface cheat sheet:**
 - Abstract class: state (fields), constructor, partial implementation.
 - Interface: pure contract, multiple inheritance, default methods.
 - **Stack for parentheses:** Always push opening brackets, pop on closing and compare.
-

13. Closing Summary

Day 4 advanced into the heart of object-oriented programming: inheritance, polymorphism, abstraction, and interfaces. Learners understood how to create class hierarchies, override methods for specific behavior, and leverage runtime polymorphism to write flexible, extensible code. Abstract classes provided a template for related classes, while interfaces enabled multiple capability contracts. The Banking Transaction Processing System case study demonstrated real-world application of these principles, allowing polymorphic processing of different account types. LeetCode problems reinforced algorithmic thinking using stacks and XOR, while HackerRank problems solidified OOP syntax.

Key Takeaways:

- Inheritance models IS-A relationships; use `extends` and `super`.
- Method overriding enables runtime polymorphism.

- Abstract classes provide partial abstraction; interfaces provide complete abstraction (with default methods in Java 8+).
- Runtime polymorphism allows a single reference to exhibit different behaviors.
- Banking system: abstract account class, specific subclasses, interfaces for optional capabilities.

End of Day 4